Guía autoguiada · AutoML en parejas (laboratorio S3.5 con AutoGluon)

PARA QUÉ SIRVE

os explica **cómo trabajar vosotros dos, paso a paso**, el laboratorio de AutoML del Sprint 3: entrenar un modelo que **predice si una reserva de TurisData se cancelará**, evaluarlo, elegir cuándo dar la alerta y ofrecerlo como servicio. Es la práctica que os prepara para el **reto «Primer modelo en producción»**.

QUÉ ES AUTOML (EN UNA FRASE)

una herramienta que **prueba varios modelos por vosotros** y os dice cuál funciona mejor. Vosotros no elegís el algoritmo: elegís **qué predecir, cómo medir si acierta y cuándo fiaros del resultado**.

- **Tiempo en clase:** unos **100 minutos** (el plan está en la sección 4). El informe del reto se remata después.
- **Coste: 0 €**. Se hace en Colab (gratis) o en vuestro ordenador. No hace falta crear cuentas de pago.
- **Qué necesitáis:** un navegador y una **cuenta de Google** (para Colab), o Python 3.10–3.13 con 8 GB de RAM si lo hacéis en local.
- **Qué no necesitáis:** saber programar a fondo. Ejecutáis celdas **de una en una**, leéis lo que sale y respondéis preguntas. Lo difícil no es el código: es **interpretar los resultados y decidir**.
- **Datos:** son **sintéticos** (inventados). Sirven para practicar, no son estadísticas reales del turismo canario.

1. Antes de empezar (10 minutos)

1. **Formad parejas** y decidid quién **conduce** primero (el otro **navega**; sección 2). Se trabaja en **una sola pantalla**.
2. **Abrid el cuaderno en Colab.** Pulsad la insignia **Abrir en Colab** del README del Sprint 3 o abrid este enlace directo.
   - **Guardad una copia suya:** *Archivo → Guardar una copia en Drive*. Así no perdéis lo que hagáis.
   - **No hace falta GPU:** el entorno de ejecución normal (CPU) vale.
3. **Ejecutad las celdas de arriba abajo, de una en una** (Mayús+Intro ejecuta la celda y pasa a la siguiente). **No uséis «Ejecutar todo»**: vais a querer leer los resultados de cada paso.
4. **La primera celda de código instala AutoGluon** y tarda **2 o 3 minutos**. Si Colab os avisa de que hay que reiniciar el entorno, aceptad y seguid desde la celda **0. Configuración**.
5. **No hace falta subir ningún archivo:** la celda **0** descarga sola `reservas_turisdata.csv` (necesita internet).
6. **Haced la prueba de arranque.** Tras ejecutar el **paso 1** debe salir:

```
(6000, 18)
```

Si sale otro número o un error, parad y preguntad antes de seguir.

Cifras de control (para saber que estáis en el laboratorio correcto):

Qué contáis	Debe salir
Reservas (filas) y columnas del CSV	**6.000** y **18**
Columnas con datos que faltan	**1**: `pais_origen`, con **138** vacíos
Proporción de reservas canceladas	**0,459** (el 45,9 %)
Reservas para entrenar / para probar	**4.800** / **1.200**

En local (si preferís vuestro ordenador): descargad la carpeta `laboratorio_automl/`, instalad con `pip install -r requirements.txt` y ejecutad el cuaderno con Jupyter o VS Code, o el script `S3.5_lab_autogluon.py`. El CSV viene dentro de la carpeta. Para un ensayo rápido: `TIME_LIMIT=15` en el script, o `LIMITE_TIEMPO = 15` en el cuaderno.

Las columnas del CSV

Son **las de la tabla `reservas`** de `turisdata.db` (mismo significado, mismos valores), **sin** `id_cliente` ni `id_alojamiento`. Las tenéis explicadas en el esquema de turisdata.db. La que **queremos predecir** es **`cancelada`** (`1` = se canceló, `0` = no). `id_reserva` solo identifica la fila: **no se usa para predecir**.

2. Cómo trabajáis en pareja: conductor y navegante

Rol	Qué hace	Qué no hace
Conductor/a	Ejecuta las celdas y cambia lo que haga falta	No decide solo: **explica en voz alta** qué hace cada celda
Navegante	Lee el «qué debe salir» de esta guía, **compara** y apunta los resultados en la hoja de registro	No dicta línea a línea ni toca el teclado

- **Cambiad de rol** cada 20 minutos o en cada bloque del plan.
- **Pedid ayuda así:** primero preguntáos entre vosotros, luego a la pareja de al lado, y **después** a vuestro docente. Llevad **la celda que habéis ejecutado y el mensaje de error**.

3. El ciclo de cada paso (5 pasos)

Repetid esto **en cada celda del laboratorio**.

1. **Leed** el título y los comentarios de la celda **en voz alta**, los dos. Cada celda tiene una pregunta o un objetivo.
2. **Predecid** antes de ejecutar: *¿qué creo que va a salir? ¿un número, una tabla, un gráfico?*
3. **Ejecutad** la celda y esperad a que termine (aparece una marca verde).
4. **Comparad** con el «Qué debe salir» de la sección 5. Los números pueden variar **un poco** de un ordenador a otro (AutoGluon usa el tiempo y los núcleos que tiene), pero **la forma** debe ser la misma.
5. **Anotad** en la hoja de registro **un dato y una frase con vuestras palabras** (qué significa). Eso será la base del informe.

4. Plan de la sesión (100 minutos)

Bloque	Pasos del cuaderno	Minutos	Qué os llevaréis
Preparar	Instalación y paso 0	10	El cuaderno funcionando
Conocer los datos	1, 2 y 3	20	Cuántas cancelan, qué falta y **por qué se parte por fechas**
Entrenar y comparar	4 y 5	15	El mejor modelo y la tabla de candidatos
Evaluar y decidir	6 y 6b	25	Precisión, recall y el **umbral elegido con costes**
Entender el modelo	7	10	Qué variables pesan más
Guardar y servir	8, 9 y 10	10	El modelo guardado y una **llamada a la API**
Cerrar	11	10	El informe de recursos y **la sesión cerrada**

Cómo encaja en el plan del día (martes 27/10): los pasos **0 a 5** son la práctica guiada (35 min) y los pasos **6 a 11** se hacen dentro del bloque del reto (80 min), que es donde se elige el umbral, se prueba la API y se cierran los recursos. Los 100 minutos de esta tabla son el total de ambos.

Si vais con prisa: poned `LIMITE_TIEMPO = 15` en el paso 0 (el entrenamiento dura unos 15 segundos) y haced **los pasos 1 a 6 y el 10**. **Si vais sobrados:** probad otro umbral con otros costes e intentad explicar a otra pareja **por qué cambia la decisión**.

5. Cada paso en una página

> En cada paso tenéis **qué hace**, **qué debe salir** (orientativo) y **qué se suele hacer mal**. Las respuestas a las preguntas están en el **solucionario paso a paso**: miradlo **después** de pensarlas vosotros. Si queréis entender **qué hace cada línea del código**, tenéis el **código explicado paso a paso**. El listado de pasos con casillas está en `listado_pasos_automl.md`.

Palabras que van a salir

Palabra	Qué significa, en sencillo
Variable objetivo	Lo que queremos predecir: `cancelada`
Entrenar	Que el modelo aprenda de reservas **pasadas** cuyo resultado ya conocemos
Datos de prueba	Reservas que el modelo **no ha visto**: sirven para saber si acierta de verdad

Palabra	Qué significa, en sencillo
Leaderboard	La tabla de modelos candidatos, del mejor al peor
Ensamblado	Un modelo que **combina varios** y reparte el voto entre ellos
Probabilidad	El modelo no dice «sí o no»: dice **«0,79 de riesgo»**
Umbral	El punto de corte: si el riesgo supera el umbral, **damos la alerta**
Precisión	De las alertas que damos, **cuántas acertaban**
Recall (exhaustividad)	De las cancelaciones que hay, **cuántas detectamos**
AUC	Nota de 0,5 (azar) a 1 (perfecto): **¿ordena bien** las reservas de más a menos riesgo?
Fuga de datos	Usar para predecir un dato que **no conoceríamos** en el momento real de decidir

Paso 0 · Configuración

Qué hace: descarga el CSV, importa las librerías y fija dos ajustes: `LIMITE_TIEMPO` (segundos de entrenamiento, **120**) y la **semilla** (42, para que los resultados se puedan repetir).

Qué debe salir: nada visible (no imprime nada).

Se suele hacer mal: saltarse esta celda o la instalación. Si luego sale `NameError` o `ModuleNotFoundError`, **empezad por aquí**.

Paso 1 · Cargar los datos

Qué hace: lee el CSV y enseña la forma de la tabla y las 5 primeras filas.

Qué debe salir: `(6000, 18)` y una tabla con `id_reserva`, fechas, `canal`, `tarifa`, `deposito`… hasta `cancelada`.

Pensad: *¿qué columnas conoceríamos el día que se hace la reserva?* (Todas, salvo `cancelada`, que es lo que predecimos.)

Paso 2 · Explorar y mirar la calidad del dato

Qué hace: muestra los tipos de dato, los nulos, la proporción de cancelaciones y un resumen de los números.

Qué debe salir: las dos fechas como `datetime`, **`pais_origen` con 138 nulos**, y `Proporción de cancelaciones: 0.459`.

Preguntas: ¿está desbalanceada la clase? ¿qué métrica conviene? ¿hay columnas con **fuga de datos**?

Se suele hacer mal: pensar que «nulos» es un error: es información (el país no se registró). AutoGluon sabe tratarlos.

Paso 3 · Partición temporal (entrenar con el pasado, probar con el futuro)

Qué hace: ordena las reservas por fecha de reserva, deja el **80 % más antiguo** para entrenar y el **20 % más reciente** para probar.

Qué debe salir: `Entrenamiento: 4800 filas (2024-01-01 a 2025-08-06)` y `Prueba: 1200 filas (2025-08-06 a 2025-12-30)`.

Pregunta: ¿por qué no repartir las reservas **al azar**?

Se suele hacer mal: quitar el `sort_values`: sin ordenar, el «pasado» y el «futuro» se mezclan y el resultado sale **demasiado optimista**.

Paso 4 · Entrenar con AutoML

Qué hace: `TabularPredictor(...).fit(...)` prueba varios modelos durante hasta `LIMITE_TIEMPO` segundos y se queda con el mejor. Sale **mucho texto**: es normal (cada modelo que prueba).

Qué debe salir: al final `Best model: WeightedEnsemble_L2` y `Tiempo de entrenamiento: 36 s` (puede ser entre 20 y 90 s). **Termina antes del límite** si ya ha probado todo. En Colab, además de los modelos de árboles, es normal que **pruebe también redes neuronales** (`NeuralNetTorch`…): saldrán más filas en el paso 5 y los números cambiarán algo.

Se suele hacer mal: interrumpir la celda por miedo a que «se ha colgado». Esperad: tarda **de medio minuto a dos minutos**. Si salen avisos (`Warning`, o `Failed to import torch`**…** en un ordenador sin esa librería), **no son errores**: AutoGluon se salta ese modelo y sigue.

Paso 5 · Leaderboard (los candidatos)

Qué hace: pone a todos los modelos a prueba con las reservas **que no han visto** y los ordena.

Qué debe salir: una tabla con unos 10 modelos (`WeightedEnsemble_L2`, `CatBoost`, `XGBoost`, `LightGBM`…). Mirad `score_test` (**~0,72** el mejor) y `score_val` (**~0,78**).

Preguntas: ¿el mejor es un ensamblado? ¿cuánto tarda cada modelo (`fit_time`)? ¿**por qué `score_val` es más alto que `score_test`**?

Se suele hacer mal: quedarse con `score_val`: es la nota «de entrenamiento». **La que vale es `score_test`**, con datos que el modelo no ha visto.

Paso 6 · Evaluar con dos umbrales

Qué hace: calcula la **probabilidad de cancelar** de cada reserva de prueba y evalúa con umbral 0,5 y con umbral 0,3. Enseña la **matriz de confusión**.

Qué debe salir (orientativo):

Umbral	Precisión	Recall	F1
0,5	~0,64	~0,60	~0,62
0,3	~0,57	~0,88	~0,69

Y `ROC-AUC: 0.722`. La matriz (umbral 0,5) se lee así:

	El modelo dijo «no cancela»	**El modelo dijo «cancela»**
Realmente no canceló	~430 (acierto)	~193 (**alerta falsa**)
Realmente canceló	~230 (**cancelación no prevista**)	~347 (acierto)

Pregunta de negocio: ¿qué cuesta más, una cancelación que no avisamos o una alerta que no hacía falta?

Se suele hacer mal: quedarse con el 0,5 «porque es el normal». **Bajar el umbral detecta más cancelaciones, pero da más alertas falsas**. Elegirlo es una decisión de negocio, no matemática.

Paso 6b · Elegir el umbral con costes (clave para el reto)

Qué hace: pone **precio** a cada tipo de error y calcula el coste total para varios umbrales. En el cuaderno hay un ejemplo con **100 €** por cancelación no prevista (noche vacía) y **40 €** por alerta falsa (por ejemplo, un descuento regalado). **Cambiadlos por vuestras hipótesis.**

Qué debe salir (con 100 € y 40 €):

Umbral	Alertas	No previstas	Falsas	Coste
0,1	1.156	4	583	23.720 €
0,2	1.044	24	491	**22.040 €**
0,3	897	68	388	22.320 €
0,5	540	230	193	30.720 €
0,7	240	389	52	40.980 €

Y debajo: **umbral teórico 0,29**, *no avisar nunca* = 57.700 € y *avisar a todos* = 24.920 €.

La regla: conviene dar la alerta cuando **riesgo > coste alerta falsa ÷ (coste alerta falsa + coste no prevista)**. Con 40 € y 100 €: 40 ÷ 140 = **0,29**.

Pensad: con estos costes, el umbral por defecto (0,5) cuesta **8.400 € más** que el 0,3. ¿Y si la alerta falsa costara solo **5 €**? (Mirad en el solucionario.)

Se suele hacer mal: decir «el mejor umbral es 0,2» sin decir **con qué costes**. El umbral **depende de los costes**: si cambian, cambia la decisión.

Paso 7 · Qué variables pesan más

Qué hace: `feature_importance` baraja cada columna y mide **cuánto empeora el modelo** sin ella: cuanto más empeora, más importante.

Qué debe salir: arriba, **`antelacion_dias`** (~0,07), **`deposito`** (~0,06), **`canal`** (~0,04) y **`tarifa`** (~0,02). El resto, casi 0.

Preguntas: ¿tienen sentido para el negocio? ¿alguna parece **fuga**?

Se suele hacer mal: pensar que «importante» significa «causa». Una variable importante **ayuda a predecir**; no demuestra que **provoque** la cancelación.

Pasos 8 y 9 · Guardar, recargar y predecir una reserva

Qué hace: guarda el modelo en la carpeta `modelo_cancelaciones`, lo **vuelve a cargar** (como haría un servicio) y define `predecir_cancelacion(reserva)`.

Qué debe salir: `Modelo recargado. Probabilidades de ejemplo: [0.457, 0.644, …]` y un ejemplo como `{'probabilidad_cancelacion': 0.457, 'alerta': False}`.

Se suele hacer mal: olvidar el paso 8 y llamar a la API sin modelo guardado: da error de carpeta no encontrada.

Paso 10 · Servir el modelo por API

Qué hace: una **API** es un servicio al que otro programa le manda una reserva y recibe la predicción. En Colab no podéis abrir la web de la API, así que la celda usa **`TestClient`**, que la llama «por dentro».

Qué debe salir:

```
{'estado': 'ok', 'modelo': 'modelo_cancelaciones'}
200 {'probabilidad_cancelacion': 0.79, 'alerta': True, 'umbral': 0.5}
```

Si cambiáis la reserva a `"noches": 0`, la API responde **422** (datos imposibles) y **no** calcula nada.

En local: `uvicorn api_cancelaciones:app --reload`, abrid `http://127.0.0.1:8000/docs` y probad `POST /predecir`. **Haced una captura** de la respuesta para el informe.

Se suele hacer mal: olvidar que el **umbral de la API es 0,5** por defecto (`UMBRAL`). Si habéis elegido otro, **tiene que reflejarse** (variable de entorno `UMBRAL`).

Paso 11 · Informe de recursos

Qué hace: suma el tiempo de entrenamiento y el tamaño del modelo en disco.

Qué debe salir: tiempo (~36 s), **~225 MB**, coste **0 €** y el recordatorio de **cerrar la sesión de Colab**.

Se suele hacer mal: no cerrar. En Colab: *Entorno de ejecución → Desconectar y eliminar entorno de ejecución*. Es la prueba de que **no queda nada abierto**.

---

6. El informe del reto: de dónde sale cada parte

El reto pide un informe de **máximo 4 páginas** con tres partes. Todo número **debe salir de vuestro experimento** (no de este documento).

Parte del informe	Qué contiene	De dónde lo sacáis
Planteamiento y datos	Qué predecimos, con qué datos, partición por fechas y por qué	Pasos 1, 2 y 3
Modelos y resultados	Candidatos comparados, métrica elegida **y por qué**, resultados en datos no vistos	Pasos 4, 5 y 6
Umbral y negocio	Umbral elegido, **costes supuestos** y una frase que entienda un directivo	Paso 6b
Qué pesa en la predicción	Variables importantes y si tienen sentido	Paso 7
Evidencia de la API	Captura de la llamada y la respuesta	Paso 10
Recursos y costes	Tabla de recursos, tiempo, coste y **prueba de cierre**	Paso 11

Tabla de recursos (rellenadla con vuestros datos):

Recurso	Cuándo se creó	Tiempo usado	Coste	Cerrado (sí/no)	Prueba
Sesión de Colab			0 €		Captura de «Desconectar»
Modelo guardado en disco			0 €		

No subáis la carpeta `modelo_cancelaciones` (pesa unos 225 MB) ni ninguna clave a vuestro repositorio.

---

7. Si algo falla

Qué veis	Qué suele pasar	Qué hacer
`ModuleNotFoundError: autogluon`	La instalación no se ejecutó o el entorno se reinició	Ejecutad **otra vez** la celda de instalación y después las demás
`NameError: name 'datos' is not defined`	Os saltasteis una celda	Ejecutad **desde el paso 0** hasta donde estabais
Falla la descarga automática del CSV (celda 0)	Sin internet o enlace caído	**Solo en este caso:** descargad `reservas_turisdata.csv` de `datos/` y subidlo al panel de archivos de Colab; la celda 0 lo detecta y no lo vuelve a descargar
La celda del paso 4 «no acaba»	Es normal que tarde	Esperad hasta 2 minutos. Para un ensayo, poned `LIMITE_TIEMPO = 15`
Colab se queda sin memoria	Sesión sobrecargada	*Entorno de ejecución → Reiniciar sesión* y ejecutad desde el paso 0
`FileNotFoundError: modelo_cancelaciones`	Falta el paso 8 o cambió la carpeta	Ejecutad los pasos 4 y 8
La API responde **422**	Un dato imposible (noches 0, precio ≤ 0…)	Es lo esperado: leed el mensaje, dice qué campo falla
Los números **no coinciden** exactamente	AutoGluon varía algo entre equipos	Es normal si el orden y la «forma» coinciden. Anotad **vuestros** números
Perdéis el trabajo	Colab desconectó la sesión	Volved a abrir la copia de Drive y ejecutad desde el paso 0

8. Al terminar

Lista de «ya sé…» (marcad entre los dos; si alguna casilla queda en blanco, repasadla)

- ☐ **Sé explicar qué hace AutoML** y qué decido yo frente a lo que decide la herramienta.
- ☐ **Sé por qué se parte por fechas** y no al azar.
- ☐ **Sé leer una matriz de confusión** y explicar precisión y recall con un ejemplo.
- ☐ **Sé elegir un umbral** pensando en costes, no en «lo normal».
- ☐ **Sé decir qué es una fuga de datos** y poner un ejemplo.
- ☐ **Sé qué es una API** y he comprobado que mi modelo responde.
- ☐ **He cerrado la sesión** y sé decir qué recursos he usado y cuánto cuestan.

Tres preguntas para hablar entre los dos (2 minutos)

1. **¿Qué paso nos costó más** y qué nos ayudó a entenderlo?
2. **Si el umbral cambia**, ¿cambia el modelo? ¿Qué cambia entonces?
3. **¿Cómo le explicaríamos al director de TurisData** por qué el umbral no es 0,5?

Antes de cerrar:

- **Guardad el cuaderno** (*Archivo → Guardar una copia en Drive*) y las **capturas** del informe.
- **Cerrad el entorno** de Colab y comprobad que lo habéis anotado en la tabla de recursos.

9. Hoja de registro de la pareja

Copiadla y rellenadla mientras trabajáis (una por pareja).

Pareja: ____ · Fecha: __

Paso	Dato que anotamos	Quién conducía	Lo que significa (con nuestras palabras)
2 · Explorar	% de cancelaciones:		
3 · Partición	Filas de entrenamiento / prueba:		
4 · Entrenar	Tiempo y mejor modelo:		
5 · Leaderboard	`score_test` del mejor:		
6 · Evaluar	Precisión y recall (0,5 y 0,3):		

Paso	Dato que anotamos	Quién conducía	Lo que significa (con nuestras palabras)
6b · Costes	Nuestros costes y umbral elegido:		
7 · Variables	Las tres más importantes:		
10 · API	Respuesta de la llamada:		
11 · Recursos	Tiempo, tamaño, coste, cerrado:		

Los tres errores en los que más caímos: 1. 2. 3.

Dudas para preguntar al docente:

Y en otras plataformas de AutoML

Lo que hacéis aquí es el mismo flujo que en una plataforma de nube (SageMaker Canvas, Vertex AI, Azure AutoML…): subir el dato, lanzar el experimento, mirar los candidatos y las métricas, elegir el umbral, publicar un endpoint y **cerrar lo que cuesta dinero**. En clase no se crean cuentas de nube. La tabla que traduce cada paso del laboratorio a esas plataformas está en el README del laboratorio.

Para repasar

Si algo de la guía no os suena, repasad los apuntes del Sprint 3 (apartado de AutoML y fuga de datos) y la ficha del reto. La guía corta de AutoGluon es el resumen de una página de este laboratorio. Los archivos del laboratorio están en `laboratorio_automl/`.